



ACIBADEM

UNIVERSITY

ACU AI Chatbot

Project Proposal Report

CSE 322: Cloud Computing

Team Name:

Acibadem Web Services

Team Members:

İsmail Esad Kılıç - 221401004
Ammar Yasir Bayır - 221401042
Vefa Mert Toker - 221401046
Yusuf Eren Arı - 221401010

GitHub Repo URL:

https://github.com/Erenimo3442/aws_acu_chatbot

Table of Contents

Project Proposal Report	1
CSE 322: Cloud Computing	1
Team Name:	1
Team Members:	1
GitHub Repo URL:	1
1. Introduction	2
Problem Statement	2
Project Objectives	2
2. Architecture	3
3. Implementation	4
Technologies	4
4. Docker Setup	5
Dockerfiles	5
Docker Compose	5
5. AI Integration	5
Planned Model and Customization Strategy	5
Planned Prompting Strategy	6
Planned Quality Assessment	6
6. Evaluation	6
7. Team Roles	7

1. Introduction

Problem Statement

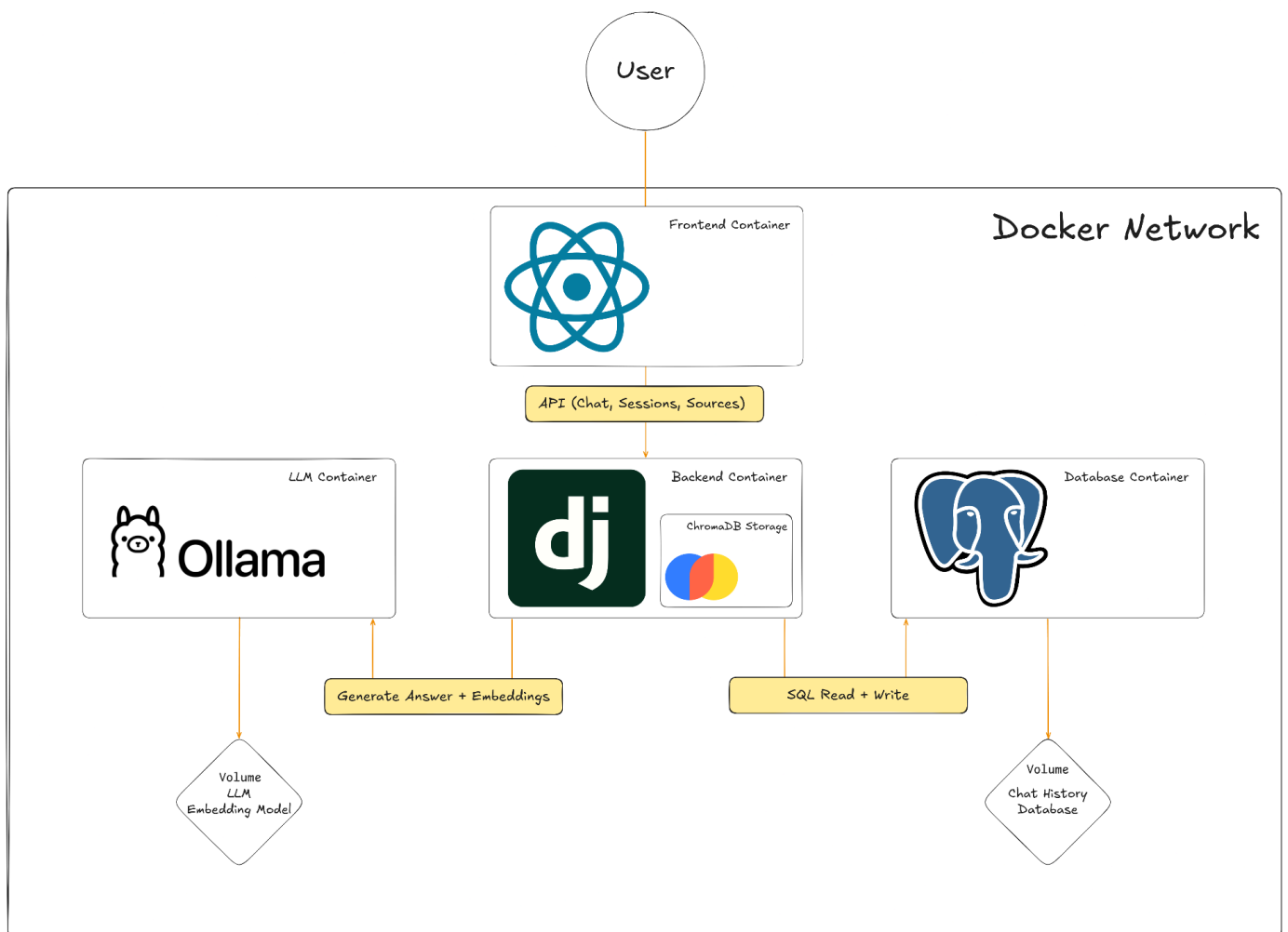
University students and applicants will need a reliable assistant that can answer academic and administrative questions from trusted institutional sources. Traditional FAQ pages are static, difficult to navigate, and often fail to provide source-grounded answers with clear traceability.

Project Objectives

This project will deliver a container-first AI chatbot backend that will:

- provide fast, source-aware answers for university-related questions,
- use retrieval-augmented generation (RAG) to reduce hallucinations,
- expose a stable API contract for chat, sessions, feedback, sources, and ingestion,
- enforce access control and rate limiting,
- support reproducible local deployment using Docker Compose.

2. Architecture



- The client will call API endpoints exposed by Django.
- Django will persist session/message/feedback data in PostgreSQL.
- Django will call Ollama for answer generation and embedding tasks.

- Django will query Chroma vector storage for semantic retrieval.
- Ingestion requests will populate and refresh vectorized content.
- All services will communicate through Docker Compose internal DNS (for example, service names such as ollama and db).

3. Implementation

Technologies

- Framework: Django (Python)
 - The team will use Django because it provides strong structure for API-oriented backends, authentication/session support, mature testing utilities, and straightforward integration with relational storage.
- Database: PostgreSQL
- PostgreSQL will be used for reliable transactional storage of sessions, messages, feedback, and API metadata.
- LLM model: qwen2.5:3b (planned default)
 - This model will be hosted via Ollama and will be selected as a practical balance between response quality and local runtime requirements.
- Serving tool: Ollama
 - Ollama will provide local model serving through a stable HTTP interface and predictable container deployment.
- RAG stack: LangChain + Chroma
 - LangChain will orchestrate model/tool calls and retrieval wiring.
 - Chroma will persist embeddings for semantic search over institutional content.

4. Docker Setup

Dockerfiles

- A multi-stage Python image build will be used to keep runtime images lean while preserving deterministic dependency installation.
- Dependencies will be installed in a builder stage and copied to the runtime stage.
- A non-root runtime user will be used for better container security.
- The backend container will expose port 8000 and run the Django application process.

Docker Compose

The project will run three core service containers:

- ollama: model serving and embeddings runtime,
- db: PostgreSQL for relational persistence,
- django-web: API backend service.

Containerization decisions the team will follow:

- Use named volumes for PostgreSQL and Ollama data persistence.
- Mount backend source during development for rapid iteration.
- Keep environment variables in a root .env file for consistent local setup.
- Use service-name networking to avoid host-specific endpoint drift.
- Keep startup orchestration explicit with depends_on, then add health-check-based readiness if needed.

5. AI Integration

Planned Model and Customization Strategy

Model selection will start with qwen2.5:3b for generation and a dedicated embedding model for retrieval.

Prompting will be grounded in a system instruction that will prioritize:

- clarity and safety,
- synthesis from retrieved context,
- source listing when evidence is available,
- no disclosure of internal tool mechanics.

Planned Prompting Strategy

The chatbot will use a structured prompt flow:

- System role: define assistant scope and safety.
- Retrieval context: inject top-k chunks with source metadata.
- Response policy: require concise answer + source section when sources exist.
- Fallback policy: if retrieval confidence is weak, return uncertainty with a safe recommendation.

Planned Quality Assessment

The team will evaluate output quality on:

- factual alignment with retrieved sources,
- citation correctness and traceability,
- completeness for multi-part questions,
- refusal behavior for unsupported or unsafe queries,
- consistency across repeated queries.

6. Evaluation

10 - 15 questions will be selected in future, to evaluate the correctness of the answers of the model. The model will be expected to provide grounded answers based on retrieved sources, with proper and verifiable citations.

- The finalized scraping strategy will be the primary router for selecting evaluation questions.
- Priority will be given to high-impact user intents and content areas that are most critical for students and applicants.
- The final question set will focus on scenarios where citation quality and factual precision matter most.

7. Team Roles

Role Slot	Member	Core Responsibilites
Backend and DevOps Lead	Eren	Django project setup, PostgreSQL schema, Docker Compose orchestration.
AI and Retrieval Specialist	İsmail	Local LLM hosting (Ollama or vLLM), prompt engineering, and AI testing.
Data and API Architect	Ammar	Web scraping scripts (BeautifulSoup or Scrapy), REST API development, and data cleaning.
UI and Integration Specialist	Vefa	Frontend chat interface (HTML or CSS or JS), chat history logic, and admin panel.